Defined in active_support/core_ext/string/filters.rb.

5.4 truncate

The method truncate returns a copy of its receiver truncated after a given length:

```
"Oh dear! Oh dear! I shall be late!".truncate(20)
# => "Oh dear! Oh dear!..."
```

Ellipsis can be customized with the :omission option:

```
"Oh dear! Oh dear! I shall be late!".truncate(20, omission:
'&hellip;')
# => "Oh dear! Oh &hellip;"
```

Note in particular that truncation takes into account the length of the omission string.

Pass a :separator to truncate the string at a natural break:

```
"Oh dear! Oh dear! I shall be late!".truncate(18)
# => "Oh dear! Oh dea..."
"Oh dear! Oh dear! I shall be late!".truncate(18, separator: ' ')
# => "Oh dear! Oh..."
```

The option :separator can be a regexp:

```
"Oh dear! Oh dear! I shall be late!".truncate(18, separator: /\s/)
# => "Oh dear! Oh..."
```

In above examples "dear" gets cut first, but then :separator prevents it.

Defined in active_support/core_ext/string/filters.rb.

5.5 truncate_words

The method truncate_words returns a copy of its receiver truncated after a given number of words:

```
"Oh dear! Oh dear! I shall be late!".truncate_words(4)
# => "Oh dear! Oh dear!..."
```

Ellipsis can be customized with the :omission option:

```
"Oh dear! Oh dear! I shall be late!".truncate_words(4, omission:
'&hellip;')
# => "Oh dear! Oh dear!&hellip;"
```

Pass a :separator to truncate the string at a natural break:

```
"Oh dear! Oh dear! I shall be late!".truncate_words(3, separator: '!')
# => "Oh dear! Oh dear! I shall be late..."
```

The option :separator can be a regexp:

```
"Oh dear! Oh dear! I shall be late!".truncate_words(4, separator:
/\s/)
# => "Oh dear! Oh dear!..."
```

Defined in active_support/core_ext/string/filters.rb.

5.6 inquiry

The inquiry method converts a string into a StringInquirer object making equality checks prettier.

```
"production".inquiry.production? # => true
"active".inquiry.inactive?       # => false
```

5.7 starts_with? and ends_with?

Active Support defines 3rd person aliases of String#start_with? and String#end_with?:

```
"foo".starts_with?("f") # => true
"foo".ends_with?("o")   # => true
```

Defined in active_support/core_ext/string/starts_ends_with.rb.

5.8 strip_heredoc

The method strip_heredoc strips indentation in heredocs.

For example in

```
if options[:usage]
  puts <<-USAGE.strip_heredoc
    This command does such and such.
```